Claude Certified Developer – Foundations (CCDV-F)

Exam · D1 / S3

53 questions · Time: 120 minutes · English

Instructions

- Each question states how many responses to select. For multiple-response items you score only with the exact set of correct options (no partial credit).
- Keep time: 120 minutes for 53 questions (about 2 min 15 s each). Flag doubtful items and return to them.
- Use only the answer sheet on the last page. Do not consult any material during the exam.
- Unofficial study material with original questions. It does not reproduce real exam items (confidential under NDA).

Domain: D1 Agents & Workflows (8) · D2 Applications & Integration (17) · D3 Claude Code (2) · D4 Eval, Testing & Debugging (1) · D5 Model Selection & Optimization (9) · D6 Prompt & Context Engineering (6) · D7 Security & Safety (4) · D8 Tools & MCPs (6)

Question 1 · **Select 3 responses.**

A team's CI calls the real Claude API on every pull request: the pipeline is slow, expensive and flaky. The team wants to test the integration code reliably. Select 3 suitable practices.

A. Use, in the PR pipeline, API client test doubles with recorded realistic responses, including tool_use, 429/529 errors and error events in the stream

B. Measure the quality of model responses in a separate evals suite, since doubles only verify the integration code and not model behavior

C. Periodically run a small contract suite against the real API to check that the doubles still reflect the real format, opening a fix task if they diverge

D. Write doubles with only a hand-made happy-path JSON, since there will never be a need to compare them with real responses

E. Keep real calls on every PR and rerun the job until it is green, since the flakiness is just noise

Question 2 · **Select 2 responses.**

Your update_ticket tool has the status parameter as a free string. In production Claude sends "Done", "finished" and "fechado", but the API only accepts open, in_progress and closed. Which changes to the tool definition address the cause? Select 2.

A. Declare status with "enum": ["open", "in_progress", "closed"] in the input_schema and explain each value in the parameter description.

B. Add input_examples with valid calls showing the accepted status values.

C. Use forced tool_choice of type tool, so Claude always calls update_ticket.

D. Cut the tool description to one short sentence, so as not to confuse the model.

E. Silently map any unknown value to "open" in the handler, so it never fails.

Question 3 · **Select 1 response.**

A dev enables thinking with display "omitted" so responses carry no reasoning text, believing this stops thinking from being billed. What actually happens to billing?

A. Reasoning is billed as input tokens, at a lower rate than output.

B. Only the visible text is billed; since the thinking field comes back empty, reasoning tokens cost nothing.

C. The thinking tokens actually generated are billed as output regardless of display; the breakdown appears in usage.output_tokens_details.thinking_tokens.

D. Only a summary of the reasoning is billed, proportional to the summarized text length.

Question 4 · **Select 1 response.**

An Agent SDK agent has three custom tools that are all lookups (lookup_customer, lookup_order, lookup_inventory). When Claude requests all three in the same turn, they run one after another and latency triples. What is the best fix?

A. Add the three tools to allowed_tools so the SDK treats them as safe for concurrent execution.

B. Mark the tools as read-only through readOnlyHint in their annotations, since custom tools run sequentially by default.

C. Lower effort to low so Claude requests fewer calls per turn.

D. Increase max_turns so Claude can spread the calls across several turns.

Question 5 · **Select 2 responses.**

In the middle of a long Claude Code conversation, the developer wants to try an alternative refactoring approach starting from the current history, without changing the original conversation, which must remain available. Select 2 suitable ways.

A. /branch, with an optional name for the new conversation.

B. claude --continue --fork-session, which copies the history into a new session ID.

C. /compact, which summarizes the history to free up space.

D. /clear, which starts with an empty context and keeps the previous history.

E. claude --resume <name>, which reopens the session and continues from it.

Question 6 · **Select 1 response.**

A service catches a generic Exception when calling the API and decides what to do with checks like `"overloaded" in str(e)`. After a wording change in error messages, the handling stopped working. What is the best approach?

A. Catch the SDK's typed exception classes, most specific first, and decide by error type/status.

B. Set max_retries=0 on the client so the SDK returns the error as raw JSON instead of raising an exception.

C. Extract the HTTP code with a regex from the error message text.

D. Add more substrings to the check to cover the new wordings.

Question 7 · **Select 2 responses.**

You extract data from contracts with the API using structured outputs. Many contracts omit the end date and the jurisdiction, and the model sometimes invents plausible values for those fields. Which schema changes reduce the problem? Select 2.

A. Put every field in required so the model always returns a value for each one.

B. Add minLength: 1 to all strings to prevent empty values.

C. Model the tipo_rescisao field as an enum that includes an explicit value such as "nao_informado", instead of a free string.

D. Leave data_termino and foro out of required, making them optional, so that missing information can be represented.

E. Rely on structured outputs to guarantee that the extracted values are true to the document.

Question 8 · **Select 1 response.**

You created the /deploy skill, which runs the production deploy procedure. You want it to run only when you type /deploy, never on Claude's own initiative because the conversation seems to call for a deploy. What should you do?

A. Set disable-model-invocation: true in the skill's frontmatter.

B. List only safe tools in the skill's allowed-tools.

C. Set context: fork in the frontmatter, to run in an isolated subagent.

D. Move the procedure to CLAUDE.md, which is only read when you ask.

Question 9 · **Select 2 responses.**

An Agent SDK agent was configured with max_turns=5 to fix tests. In production, some runs end with no final text at all. Select 2 correct statements about this behavior and how to handle it.

A. The SDK automatically doubles max_turns and continues the loop when the limit is reached.

B. The ResultMessage still carries session_id, so the session can be resumed with a higher limit to continue the work.

C. The run ends with a ResultMessage of subtype error_max_turns and no result field; code must check the subtype before reading result.

D. Raising max_budget_usd also raises the turn limit, so a larger budget is enough to let the loop run longer.

E. max_turns counts only turns that call tools; the final text-only response never consumes a turn.

Question 10 · **Select 1 response.**

A bank will launch a Claude-based support chatbot. Compliance requires that "every answer given to a customer be auditable". Which design decision best meets this requirement?

- **A.** Persist, per response, the conversation id, prompt version, model used, the request-id returned by the API and the messages exchanged, with a defined retention policy
- **B.** Log only the final text shown to the customer, to reduce stored volume and the risk of retaining sensitive data
- **C.** Enable streaming, since the SSE events received by the customer's browser already form an auditable record of the interaction
- **D.** Rely on logs kept by Anthropic and request them from the provider whenever an internal or external audit takes place

Question 11 · **Select 2 responses.**

The team wants a /release procedure that runs only when someone types the command (never by Claude's own decision) and whose long reference material (checklist and templates) does not take up context in every session. Select 2 suitable practices.

- **A.** Paste the full procedure and templates into the project's CLAUDE.md.
- **B.** Set user-invocable: false so the skill is only used deliberately.
- **C.** Use allowed-tools to control when Claude can trigger the skill.
- **D.** Create .claude/skills/release/SKILL.md with disable-model-invocation: true.
- **E.** Keep the checklist and templates in separate files inside the skill folder, referenced from SKILL.md.

Question 12 · **Select 3 responses.**

A nightly job sends 50k requests to the Message Batches API, all with the same 30k-token document as prefix and different questions. There is no urgency for responses. Select 3 correct practices to minimize cost.

- **A.** Inside the batch, send a request with max_tokens 0 to pre-warm the cache before the others.
- **B.** Include identical cache_control blocks in every request of the batch to raise the chance of cache hits.
- **C.** Count on guaranteed hits, since the batch processes requests sequentially in order.
- **D.** Consider the 1-hour TTL, since batches can take longer than 5 minutes to process.
- **E.** Use the Batches API, which gives a 50% discount for non-urgent asynchronous work.

Question 13 · **Select 2 responses.**

Your team enabled 12 plugins in Claude Code and notices sessions start with less context space, even on tasks that use few of them. Which statements about plugin cost are true? Select 2.

- **A.** You can stop the cost without uninstalling by disabling the plugin with /plugin or with claude plugin disable.
- **B.** The name and description of each skill Claude can invoke on its own sit in context every turn, even in sessions where nothing from the plugin is used.
- **C.** Uninstalling is the only way to remove a plugin's context cost.
- **D.** A plugin only affects the session when one of its skills is invoked; hooks and MCP servers stay inactive until then.
- **E.** The full text of all skills from enabled plugins is loaded at the start of every session.

Question 14 · **Select 3 responses.**

A dev is estimating how much of the context window a request for an agent consumes. The request has a system prompt, 12 tool definitions, a history with tool_result blocks, and the text Claude will generate in the turn. Which statements about what counts toward the context window are correct? Select 3.

- **A.** The content of tool_result blocks in the history counts like any other message.
- **B.** The text Claude generates in the turn, including thinking, counts toward the window.

C. Tool definitions count toward the window, just like the system prompt.

D. Tokens read from a cached prefix (prompt caching) no longer occupy the window; caching only changes the price.

E. Only user/assistant messages count; the system prompt and tool definitions are left out of the total.

Question 15 · **Select 2 responses.**

A team wants to reduce the risk of leaking Claude API keys before any incident happens. Which two prevention practices are recommended? Select 2.

A. Keep the same key without rotation until a leak is suspected.

B. Regularly scan repositories with a tool such as Gitleaks.

C. Encode the key in base64 in the front-end bundle so it is not readable.

D. Share a single team key over chat to speed up onboarding.

E. Load the key through an environment variable and list .env in .gitignore.

Question 16 · **Select 1 response.**

A prompt pastes three contracts as plain text separated by '---' lines, followed by the instructions. Claude sometimes attributes a clause to the wrong contract and mixes contract passages with the instructions. What is the best fix?

A. Move the three contracts into the system prompt so they carry more authority than the instructions.

B. Wrap each contract in <document index="n"> with <source> and <document_content>, inside <documents>.

C. Repeat the instructions after each contract so Claude does not confuse them with the text.

D. Replace the '---' separators with uppercase banners like '=== CONTRACT 2 ==='.

Question 17 · **Select 1 response.**

A support agent reads customer tickets and has the query_db tool with a database administrator credential and the send_email tool with no recipient restriction. The team fears a malicious ticket could make the agent exfiltrate data. What is the best approach?

A. Politely ask, in the test tickets, that the agent ignore instructions contained in the customer text.

B. Add 'never send data to third parties' to the system prompt and keep the current permissions.

C. A read-only credential on the needed tables, narrow tools, and no unnecessary secrets.

D. Log all tool calls for auditing and review them weekly, keeping the permissions.

Question 18 · **Select 2 responses.**

Your team has a lookup_customer function that queries the CRM. Today only one internal service, which calls the Messages API, uses it. Soon it will also be used in Claude Code, Claude Desktop and another app. Which reasons justify packaging it as an MCP server? Select 2.

A. Several hosts (Claude Code, Claude Desktop, other apps) can connect to the same server instead of each reimplementing the tool.

B. Tools defined in application code cannot reach internal APIs; only MCP servers can.

C. The Messages API only accepts tools that come from MCP servers.

D. The connection and authentication to the CRM live in one place, handled by the server, and are not duplicated in each client.

E. A tool exposed via MCP is called by the model more accurately than a tool defined in the application code.

Question 19 · **Select 2 responses.**

An assistant answers questions about an annual report and sometimes confidently states figures that are not in the report. Which two measures help reduce and detect this? Select 2.

- **A.** Run the same prompt several times and review the answers that disagree, as they may indicate hallucination.
- **B.** Instruct Claude to fill in gaps with general knowledge when the report lacks the data.
- **C.** Drop human validation once answers are restricted to the report.
- **D.** Ask Claude to report a confidence from 0 to 100 and automatically accept answers above 90.
- **E.** Give explicit permission to answer that there is not enough information when the report lacks the data.

Question 20 · **Select 2 responses.**

A support chatbot requirement says only: "must be fast and always available". Before designing, the team must turn it into verifiable requirements. Select 2 suitable approaches.

- **A.** Set 100% availability as the goal, with no error response tolerated in any period
- **B.** Define p95 time-to-first-token below an agreed value, under a specified number of concurrent sessions, measured in production
- **C.** Require average latency below the target, since the mean summarizes every user's experience well
- **D.** Define an availability SLO (for example, % of requests served successfully per month) and the expected degraded behavior when the API returns 429 or 529
- **E.** Always pick the largest available model, since that guarantees fast responses and availability

Question 21 · **Select 1 response.**

The repository has a committed .claude/settings.json with permissions.allow ["Bash(npm run lint)"] and permissions.deny ["Read(./.env)"]. A new person clones the repo and opens Claude Code, without having trusted the folder yet. What happens?

- **A.** Neither applies until trust is granted, so .env can be read in the meantime.
- **B.** The allow rule applies right away, while deny only applies after trusting the folder, since it requires confirmation.
- **C.** The deny rule applies right away; the allow rule only takes effect after trusting the folder, so lint still prompts.
- **D.** Both rules apply immediately, since the file is committed to the repository.

Question 22 · **Select 1 response.**

Your company wants a read-only MCP server over the ERP, used by hundreds of devs in Claude Code (and by other apps), identifying the user on every access. Which design fits best?

- **A.** A remote server using SSE transport, as it is the most recommended for remote services.
- **B.** A single stdio process on a central host, to which all devs connect at the same time.
- **C.** A remote server using Streamable HTTP transport, standard HTTP authentication (OAuth recommended by MCP for obtaining tokens) and clients connecting by URL.
- **D.** A stdio server that each dev runs locally, with the ERP credential in an environment variable.

Question 23 · **Select 1 response.**

An internal tool calls Claude once every ~30 minutes with the same large prefix and cache_control at the default TTL. The bill went up after enabling caching, and the response shows high cache_creation_input_tokens and cache_read_input_tokens always 0. What is the best interpretation and action?

- **A.** The prefix is too small, so nothing is read; raising max_tokens fixes it.
- **B.** The default TTL (5 minutes) expires before the next call (~30 minutes): every request pays the cache write and never reads; the action is to remove caching or evaluate the 1-hour TTL, which covers that gap between calls.

C. The usage fields are wrong on recent models; caching is working and billing will stabilize.

D. You need to add more breakpoints (up to 4) so the cache starts being read.

Question 24 · **Select 1 response.**

A team keeps an extractor's system prompt in a repository and edits it in place under the label "v2". A "small" rewording changed the output format, broke the downstream parser and went to 100% of traffic at once, even though the offline evals passed. The team decides to adopt semantic versioning (SemVer) for its prompts. Which versioning and rollout approach is best?

A. Pick the version number by diff size: few changed lines mean PATCH, even if the output format changes

B. Release straight to 100% once offline evals pass, since they replace observation in production

C. Keep the "v2" label and overwrite it on every tweak, since Git history already lets you go back to any previous version

D. Declare the output format as the prompt's public API, treat published versions as immutable (any change becomes a new version; broken format = MAJOR), gate with evals, then release as a canary with metrics and rollback

Question 25 · **Select 2 responses.**

A review subagent was defined in .claude/agents/reviewer.md, with description "Read-only reviewer; never modifies files" and no tools field. It still used Write and Edit. The team requires those two tools to be unavailable to it. Select 2 effective changes.

A. Add disallowedTools: Write, Edit to the subagent's frontmatter.

B. Set isolation: worktree so edits never reach the main checkout.

C. Reinforce in the description field that the subagent must never use Write or Edit.

D. Add tools: Read, Grep, Glob to the subagent's frontmatter.

E. Set model: haiku, since a smaller model tends to edit less.

Question 26 · **Select 1 response.**

A FAQ app on a Sonnet-tier model uses adaptive thinking. Monitoring shows thinking blocks on many trivial questions, adding latency and output cost. Adding "don't think, answer directly" to the system prompt did not reduce it reliably. What is the best approach?

A. Set a small budget_tokens in thinking to cap reasoning at a few tokens.

B. Lower the effort level (output_config.effort), measure the quality impact, and only then add prompt guidance if still needed.

C. Strengthen the system prompt ("NEVER think") and assume the model will always comply.

D. Raise max_tokens so thinking finishes faster and frees the answer.

Question 27 · **Select 1 response.**

A team creates a PreToolUse hook that detects when Claude Code tries to read a .env file, prints a message to stderr, and ends with exit 1. The file is still read anyway. What is the best fix?

A. Keep exit 1, which, being the conventional Unix failure code, already blocks the action.

B. Move the logic to a PostToolUse hook, which runs after the tool has executed.

C. End the script with exit 2: in PreToolUse this blocks the tool call and the stderr becomes the block message.

D. Print the message to stdout and end with exit 0, since the message alone cancels the tool call.

Question 28 · **Select 2 responses.**

The get_invoice(invoice_id) tool runs in your backend, with strict: true and invoice_id as a string in the schema. Another tool returns, in a tool_result, the text of a third-party email that tries to induce Claude to request ANOTHER customer's invoice. Which statements are true? Select 2.

A. strict: true guarantees the invoice_id sent belongs to the authenticated customer.

B. strict: true guarantees the tool's inputs match the declared schema.

C. Untrusted content should be placed in the system prompt, which is the safest place for it.

D. Content in a tool_result coming from outside sources, such as emails, must be treated as untrusted, since it can embed instructions that try to redirect Claude (indirect prompt injection).

E. The email is inside a tool_result, so its content is trustworthy because it comes from your own backend.

Question 29 · **Select 1 response.**

Agent trace: on turns 1, 2 and 3 Claude calls search_docs with {"query":"rate limits"} and gets is_error=true: "Missing required 'repo' parameter". On turn 4 it apologizes and gives up. The search_docs schema requires repo, but the description is "Search docs." and repo has no description. What is the best fix?

A. Increase max_turns so Claude eventually gets the parameter right in more attempts.

B. Increase retries and backoff in the tool executor, since the integration is failing.

C. Switch to forced tool_choice on search_docs, since Claude picked the wrong tool.

D. Expand the description of the tool and of the repo parameter (what it is, valid values) and enable strict: true, treating it as an under-specified definition.

Question 30 · **Select 1 response.**

Your app uses the Messages API with a client tool issue_refund, which moves money. You want a human to approve each refund before it happens. Which approach?

A. Turn issue_refund into a server tool, so that execution and approval are handled by Anthropic.

B. In your code that handles the tool_use block, pause before executing, ask for approval and, if denied, return a tool_result with is_error true explaining the denial.

C. Let the handler execute the refund and, if the human disagrees afterwards, ask Claude to reverse it.

D. Write "always ask the user for approval before calling" in the tool description and trust Claude to comply.

Question 31 · **Select 1 response.**

An app streams a response in which Claude calls a tool. The code runs json.loads on each partial_json received in input_json_delta events and throws on almost every event. What is the correct approach?

A. Catch the exception and use the last fragment that parsed successfully as the tool arguments

B. Accumulate fragments per block index, parse on receiving content_block_stop, and only then run the tool

C. Run the tool as soon as the first field shows up in the fragment, using a regex, to reduce latency

D. Turn streaming off only during tool use, since there is no way to get the arguments JSON from streamed requests

Question 32 · **Select 1 response.**

A team uses a single Claude call to review code changes for vulnerabilities. The problem is variance: across repeated runs, real findings sometimes show up and sometimes disappear. Which workflow pattern addresses this problem?

A. Voting: run the same review several times independently and flag the change when enough runs report a problem.

B. Routing: classify each change by language and send it to a specialized review prompt.

C. Prompt chaining: pass the review through a second prompt that rewrites and refines the report text.

D. Sectioning: split the diff into independent parts and process each part in parallel, each with a different prompt.

Question 33 · **Select 2 responses.**

An application uses a model with a 1M-token context window. Each request sends a 300k-token prefix marked for prompt caching, plus the user question, with adaptive thinking on and max_tokens set. A colleague makes claims about context window usage. Select 2 correct statements.

A. Generated thinking tokens count toward the context window and are part of the max_tokens limit.

B. The window limits only input; output tokens are counted against a separate limit.

C. The 300k cached prefix tokens still occupy the context window; caching changes their price, not the space they take.

D. Tokens read from cache do not count toward the window because they are not reprocessed as normal input.

E. If the input alone exceeds the window, the API silently truncates the oldest messages and responds normally.

Question 34 · **Select 2 responses.**

Before sending large prompts, your internal API calls the token counting endpoint to decide routing and check limits. Select 2 correct statements about this endpoint.

A. It returns an estimate; the actual input token count at message creation may differ by a small amount.

B. It returns the exact number that will be billed, always identical to the real response usage.

C. It is free to use, subject to its own requests-per-minute limits, independent of message creation limits.

D. It honors cache_control blocks, so the result already reflects the cache read discount.

E. It accepts the same request as the Messages API, including server tools such as web search.

Question 35 · **Select 1 response.**

An Agent SDK agent received "always write release notes in Portuguese" in its initial prompt. In long sessions, after compact_boundary events, it starts writing in English. What is the best fix?

A. Set effort to max so the model remembers the initial instructions better.

B. Move the rule into a CLAUDE.md loaded through setting_sources, which is re-injected on every request.

C. Raise max_budget_usd so the original context is not summarized.

D. Resume the session with session_id on every turn to restore the original prompt.

Question 36 · **Select 1 response.**

In an agent loop, a response arrives with stop_reason "max_tokens" and the last content block is a tool_use whose JSON input looks cut off mid-way. What should you do?

A. Treat the response as incomplete and retry the request with a higher max_tokens instead of running the tool.

B. Send the partial tool_use back as a tool_result and ask the model to complete the JSON.

C. Run the tool with the partial input, assuming the fields that arrived are enough.

D. Ignore the stop_reason, since the SDK automatically completes truncated tool calls.

Question 37 · **Select 3 responses.**

A Node service processes 2,000 documents by firing all API calls with Promise.all and receives a flood of 429 errors. Select 3 suitable fixes.

A. Ramp load up gradually instead of jumping from zero to peak, which also avoids acceleration limits

B. Retry the request immediately in a loop, until one of the attempts goes through

C. Add independent retries at every layer (queue, service and client), so each failure gets more chances to pass

D. Limit concurrency with a worker pool or semaphore, sized below the organization's rate limits

E. Apply retry with exponential backoff and jitter, honoring the retry-after header when present and with a maximum number of attempts

Question 38 · **Select 1 response.**

An Agent SDK agent will carry out a large refactor. The team wants it to read the code and propose a plan for human approval before modifying any file. Which permission_mode fits?

A. plan, where Claude explores and plans, and edits made with its file tools go to the canUseTool callback.

B. dontAsk, which never prompts, so nothing unreviewed ever gets executed.

C. bypassPermissions, which runs calls without asking for approval and is recommended for local testing.

D. acceptEdits, which approves edits automatically to speed up iteration.

Question 39 · **Select 1 response.**

In your team's Claude Code there are two instructions: (1) "tests run with pnpm test and API handlers live in src/api/handlers/"; (2) "short answers, with little explanation", which some devs want to turn off while learning the code. Where should each go?

A. (1) in a skill with disable-model-invocation; (2) in a SessionStart hook.

B. (1) in CLAUDE.md; (2) in an output style, which can be switched whenever wanted.

C. (1) in an output style; (2) in CLAUDE.md.

D. Both in CLAUDE.md, since it loads every session and covers any kind of instruction.

Question 40 · **Select 1 response.**

A team will migrate from an older-generation model to a newer-generation tier while keeping the same prompts. Their cost budget and "fits in the window" checks use token counts measured on the old model. What is the best approach?

A. Reuse the counts, since the number of tokens for a text is the same on every Claude model.

B. Re-evaluate only if the price per million tokens changes; otherwise the budget stays valid.

C. Recount the prompts with the token counting endpoint using the target model and redo cost and window estimates, plus run evals.

D. Assume newer models always produce fewer tokens for the same text and lower the budget.

Question 41 · **Select 1 response.**

A service uses output_config.format with a JSON schema and writes Claude's output straight to the database. In production, parsing sometimes fails on responses cut off midway. What is the best approach?

A. Apply a function that "fixes" truncated JSON by closing braces and quotes, and write the repaired result

B. Treat the output as always valid, since structured outputs guarantee the schema on any response

C. Remove the schema and start the assistant turn with "{" to force JSON format on the newest models

D. Check stop_reason before parsing, handle max_tokens (retry with a higher limit) and refusal as separate cases, and validate at the boundary before writing

Question 42 · **Select 1 response.**

In a long Claude Code session you ask for a formatting tweak. Claude gets it wrong, you correct it, and it gets the same thing wrong again. What is the best next action?

A. Run /clear and start with a single new prompt that already folds in what you learned from the attempts.

B. Make a third correction, writing the instruction in capital letters for emphasis.

C. Add the sentence to CLAUDE.md in ALL CAPS with IMPORTANT and keep going in the same session.

D. Keep correcting in the same conversation, since more history of attempts gives the model more context.

Question 43 · **Select 3 responses.**

A Message Batches API job with 5,000 requests finished with 4,700 succeeded, 200 errored and 100 expired results. Select 3 correct practices to consolidate and recover the work.

A. Resubmit expired items and server-error items in a new batch, and first fix items with invalid_request_error

B. Match each result to its input by line position in the results file, since it follows submission order

C. Match each result to its source input by custom_id, since result order is not guaranteed

D. Count on errored and expired items not being billed, so reprocessing them does not pay twice for the same work

E. Resubmit the entire 5,000-item batch to ensure all results come from the same processing run

Question 44 · **Select 1 response.**

In a chat with Claude, the dev calls the Messages API on each turn sending only the new user message. The assistant "forgets" everything said before. What is the most appropriate fix?

A. Increase max_tokens, so the model can retain more context from one call to the next

B. Send only the user's earlier turns on each call, omitting the assistant's replies to save tokens

C. Reuse the same request-id header on every turn, so the API associates the calls with the same conversation

D. Store the conversation's message list and send it in full on every call, appending the assistant's reply and the new user turn

Question 45 · **Select 1 response.**

A triage service uses output_config.format with a JSON schema whose "category" field is an enum with the values "Billing", "Technical support" and "Other". The code compares the received category by exact equality and occasionally falls into the "unknown" branch because, for example, "Technical Support" arrived. What is the best fix?

A. Compare the value case-insensitively (normalizing first) and avoid enum values that differ only in capitalization.

B. Abandon structured outputs and ask for free-text JSON, losing the structure conformance the schema guarantees.

C. Assume the schema guarantees the exact enum spelling and treat the mismatch as an API bug to report.

D. Treat it as a transient error and retry the request until the value comes back exactly as in the schema.

Question 46 · **Select 1 response.**

A pipeline generates contract summaries. The team wants to log the draft, evaluate the review of the draft against criteria and, between steps, decide whether to proceed to refinement or divert to human review. What is the best approach?

A. Split into sequential calls (draft, review, refine) and branch on the output of each step.

B. A single prompt that asks for draft, review and final version, returning only the final version to save tokens.

C. Raise the effort of the single call so the internal review is more rigorous, and skip logging the steps.

D. Rely on adaptive thinking to do the three steps internally, since the model reasons through multiple steps on its own.

Question 47 · Select 1 response.

A web chat shows Claude's reply token by token. The user sends each question through a normal POST request, and the server only needs to push text back during generation. Which server-to-browser transport is the best choice?

- **A.** Call Anthropic's API directly from the browser with the key embedded in JavaScript, removing the intermediate server
- **B.** Polling an endpoint every 100 ms, fetching the newly accumulated tokens
- **C.** WebSocket, because streaming responses requires a bidirectional channel between browser and server
- **D.** Server-Sent Events over HTTP, since the flow is server-to-client only and the browser reconnects automatically

Question 48 · Select 1 response.

Your team wants everyone in the repository to start Claude Code with the same default model, and for changes to that choice to go through code review and be revertible. Which approach?

- **A.** Each dev runs /model and saves the choice, which is written to each person's ~/.claude/settings.json.
- **B.** Write a line in the project's CLAUDE.md asking Claude to use the desired model.
- **C.** Set the model key in .claude/settings.json and commit it to the repository.
- **D.** Set the model key in each dev's .claude/settings.local.json.

Question 49 · Select 1 response.

Claude's output goes to an SMS gateway that breaks on markdown. The system prompt says 'NEVER use markdown' and asterisks still show up now and then. What is the best improvement?

- **A.** Move the prohibition to the end of each user message and remove it from the system prompt.
- **B.** Add a rule saying formatting mistakes will be penalized and the response will be rejected.
- **C.** Write the same prohibition three times in uppercase to reinforce its priority.
- **D.** Explain the reason: the reply goes out by SMS and markdown shows up literally to the user.

Question 50 · Select 3 responses.

An agent that summarizes web pages and emails from unknown senders is going to production. Which three practices are consistent with defending against indirect injection? Select 3.

- **A.** Test before deployment with documents, emails and tool outputs that contain deliberate injection attempts.
- **B.** Insert the page text into the system prompt so Claude gives the retrieved content more weight.
- **C.** Give the agent only the tools and data it needs, running them in an isolated environment with restricted permissions.
- **D.** Skip adversarial testing, since Claude is inherently resilient to this kind of attack.
- **E.** Continuously analyze outputs for signs of successful injection and use that to refine prompts and filters.

Question 51 · Select 1 response.

You want to try a plugin (with skills and hooks) only in this repository, without affecting your other projects or the teammates who use the same repo. Which install scope should you pick?

- **A.** User scope, which enables it for you in every project on this machine.
- **B.** Managed scope, set by the organization for all developers.
- **C.** Project scope, recording it in the committed .claude/settings.json and asking teammates to ignore it.
- **D.** Local scope, which enables it for you only in this repository.

Question 52 · **Select 1 response.**

A three-call pipeline extracts invoice fields, reconciles them with the ERP and drafts a collection email. Sometimes extraction returns empty required fields and the final email comes out confident but wrong. What is the best approach?

A. Add "be careful if the data is incomplete" to the email prompt and keep the rest of the flow unchanged.

B. Merge the three steps into one larger prompt so the model sees the whole task at once.

C. Add a programmatic check (gate) after extraction that validates the required fields and stops or retries the step before moving on.

D. Replace the pipeline with an autonomous agent that decides on its own which steps to run and in what order.

Question 53 · **Select 1 response.**

An agent built with the Claude Agent SDK exposes a custom get_order tool through an in-process MCP server registered in mcpServers under the key "shop". Claude sees and uses the tool, but in unattended runs every call stalls waiting for a permission approval. Which change lets the calls run without a prompt?

A. Mark the tool with readOnlyHint: true in its annotations, since read-only tools skip approval.

B. Rewrite the tool description to say it is safe so Claude does not ask for approval.

C. Pass tools: ["get_order"] in the options so the tool is both available and approved.

D. Add the fully qualified name mcp__shop__get_order (or the wildcard mcp__shop__*) to allowedTools.

Answer sheet

1	(A)	(B)	(C)	(D)	(E)
2	(A)	(B)	(C)	(D)	(E)
3	(A)	(B)	(C)	(D)	(E)
4	(A)	(B)	(C)	(D)	(E)
5	(A)	(B)	(C)	(D)	(E)
6	(A)	(B)	(C)	(D)	(E)
7	(A)	(B)	(C)	(D)	(E)
8	(A)	(B)	(C)	(D)	(E)
9	(A)	(B)	(C)	(D)	(E)
10	(A)	(B)	(C)	(D)	(E)
11	(A)	(B)	(C)	(D)	(E)
12	(A)	(B)	(C)	(D)	(E)
13	(A)	(B)	(C)	(D)	(E)
14	(A)	(B)	(C)	(D)	(E)
15	(A)	(B)	(C)	(D)	(E)
16	(A)	(B)	(C)	(D)	(E)
17	(A)	(B)	(C)	(D)	(E)
18	(A)	(B)	(C)	(D)	(E)
19	(A)	(B)	(C)	(D)	(E)
20	(A)	(B)	(C)	(D)	(E)
21	(A)	(B)	(C)	(D)	(E)
22	(A)	(B)	(C)	(D)	(E)
23	(A)	(B)	(C)	(D)	(E)
24	(A)	(B)	(C)	(D)	(E)
25	(A)	(B)	(C)	(D)	(E)
26	(A)	(B)	(C)	(D)	(E)
27	(A)	(B)	(C)	(D)	(E)

28	(A)	(B)	(C)	(D)	(E)
29	(A)	(B)	(C)	(D)	(E)
30	(A)	(B)	(C)	(D)	(E)
31	(A)	(B)	(C)	(D)	(E)
32	(A)	(B)	(C)	(D)	(E)
33	(A)	(B)	(C)	(D)	(E)
34	(A)	(B)	(C)	(D)	(E)
35	(A)	(B)	(C)	(D)	(E)
36	(A)	(B)	(C)	(D)	(E)
37	(A)	(B)	(C)	(D)	(E)
38	(A)	(B)	(C)	(D)	(E)
39	(A)	(B)	(C)	(D)	(E)
40	(A)	(B)	(C)	(D)	(E)
41	(A)	(B)	(C)	(D)	(E)
42	(A)	(B)	(C)	(D)	(E)
43	(A)	(B)	(C)	(D)	(E)
44	(A)	(B)	(C)	(D)	(E)
45	(A)	(B)	(C)	(D)	(E)
46	(A)	(B)	(C)	(D)	(E)
47	(A)	(B)	(C)	(D)	(E)
48	(A)	(B)	(C)	(D)	(E)
49	(A)	(B)	(C)	(D)	(E)
50	(A)	(B)	(C)	(D)	(E)
51	(A)	(B)	(C)	(D)	(E)
52	(A)	(B)	(C)	(D)	(E)
53	(A)	(B)	(C)	(D)	(E)